
PROGRAMAÇÃO ORIENTADA A OBJETO (POO)

PYTHON – FUNÇÕES E CLASSES

PROF: HELENA

@HelenaNascimento

Bacharel em SI

2 artigos publicados

Conhecimento Avançado:

- **Análise de Dados**
- **Automação de Processos**
- **Banco de Dados**



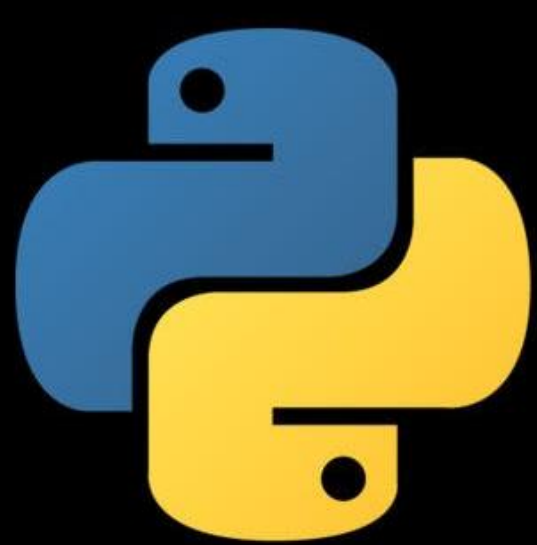
hnasc1989



@hnasc.1989



HelenaNascimento



pythonTM

-
- Funções (def):
 - Abstração
 - Programação Estruturada
 - Definindo funções
 - Resultado de funções
 - Variáveis locais e globais
 - Argumentos de funções
 - Argumentos default

- Classes (class) – Part 1:
 - Orientação a Objeto
 - Objetos em Python
 - Abstração

Antes de falarmos sobre POO, vamos falar mais um pouco sobre estrutura de dados



FUNÇÕES

Abstração:

Processo de ocultar detalhes de implementação e expor apenas o que é necessário para o uso.

```
print('olá')
```

Funções e Programação Estruturada:

Programação Estruturada:

É um paradigma baseado em:

- Sequência
- Decisão(if)
- Repetição (for, while)
- Funções

Funções Permitem:

- Modularização do código
 - Reutilização
 - Melhor legibilidade
 - Manutenção Facilitada
-

Estrutura de uma função:

python

```
def soma(a, b):  
    return a + b
```

```
resultado = soma(5, 3)  
print(resultado) # 8
```

#Estrutura de uma Função em Python

```
def nome_da_funcao(parametro1, parametro2):  
    #Corpo da Função  
    resultado = parametro1 + parametro2  
    return resultado #Retorna o valor da soma
```

```
print(nome_da_funcao(5, 10)) #Chama a função e imprime o resultado
```

O que é uma função?

Função é um bloco de código reutilizável que executa uma tarefa específica e pode receber dados (parâmetros) e retornar um resultado.

Variáveis Globais:

Criada fora da função

Pode ser acessada dentro da função

```
y = 20

def mostrar():
    print(y)

mostrar()
```

Exemplo:

```
contador = 0

def incrementar():
    global contador
    contador += 1
```

Funções com argumentos:

Os argumentos são definidos dentro dos parênteses na criação da função.

```
def saudar(nome):  
    print(f"Olá, {nome}! Tudo bem?")  
  
# Chamando a função e passando o valor  
saudar("Felipe")  
saudar("Maria")
```

Eles também podem ser múltiplos:

```
def calcular_area_retangulo(base, altura):  
    area = base * altura  
    return area  
  
resultado = calcular_area_retangulo(10, 5)  
print(f"A área é: {resultado}")
```

Argumentos com Valor Padrão (Default)

Às vezes, você precisa que um argumento tenha um valor pré-definido caso o usuário não envie nada.

```
def exibir_mensagem(texto, autor="Anônimo"):
    print(f'"{texto}" - escrito por {autor}')

exibir_mensagem("A persistência é o caminho do êxito.") # Usa o padrão
exibir_mensagem("Penso, logo existo", "Descartes")      # Sobrescreve o padrão
```

Argumentos Arbitrários (*args e **kwargs)

Se você não souber quantos argumentos serão passados, use estas sintaxes:

*args: Recebe múltiplos valores como uma **tupla**.

kwargs: **Recebe múltiplos valores nomeados como um **dicionário.

```
# Exemplo com *args (Soma quantos números forem enviados)
def somar_todos(*numeros):
    return sum(numeros)

print(somar_todos(1, 2, 3, 4, 5))

# Exemplo com **kwargs (Recebe dados estruturados)
def salvar_perfil(**dados):
    for chave, valor in dados.items():
        print(f"{chave}: {valor}")

salvar_perfil(usuario="dev_python", nivel="Sênior", ativo=True)
```

Resumo dos tipos de argumentos:

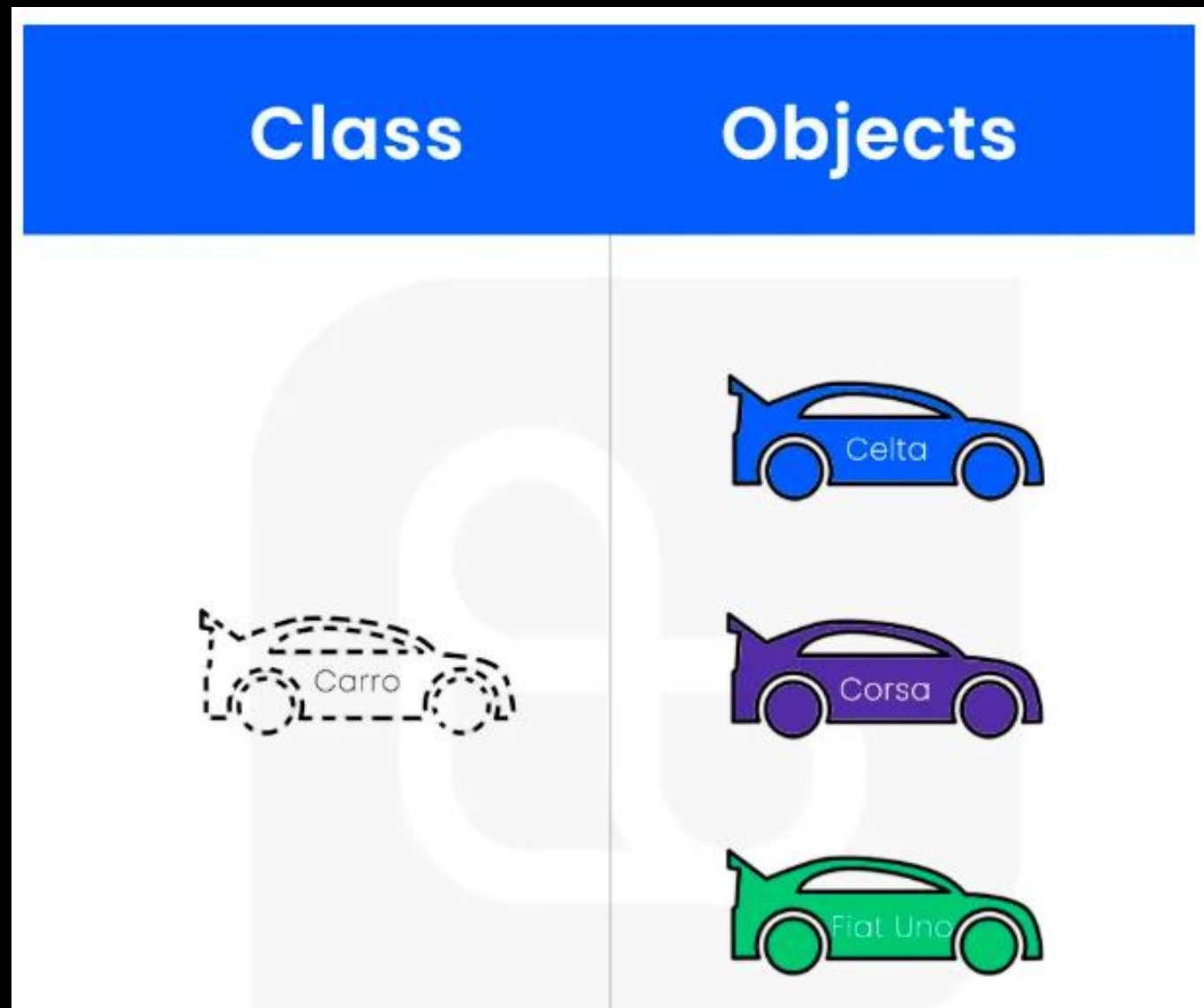
Tipo	Exemplo	Quando usar?
Posicional	<code>func(a, b)</code>	Quando a ordem dos dados é fixa.
Nomeado	<code>func(nome="Ana")</code>	Para deixar o código mais legível.
Padrão	<code>func(país="Brasil")</code>	Quando um valor se repete na maioria das vezes.
Dinâmico	<code>func(*args)</code>	Quando o número de entradas é imprevisível.



PROGRAMAÇÃO ORIENTADA A OBJETO

Definição de Classe e Objeto: "Uma classe é uma estrutura que define as propriedades (atributos) e comportamentos (métodos) que os objetos criados a partir dela terão."

1. Classe – Molde ou planta do objeto.
2. Objeto – Instância criada a partir da classe.
3. Atributos – Características do objeto (dados).
4. Métodos – Ações que o objeto executa.
5. Encapsulamento – Controle de acesso e organização do código.



Exemplo:

```
1 class Aluno:
2     def __init__(self, nome, idade):
3         self.nome = nome
4         self.idade = idade
5         self.notas = []
6
7     def adicionar_nota(self, nota):
8         self.notas.append(nota)
9
10    def calcular_media(self):
11        if self.notas:
12            return sum(self.notas) / len(self.notas)
13        else:
14            return 0
```

Nome da Classe

Atributos da Classe

Método da Classe

Método da Classe

Criando Objetos

```
18 # Criando objetos
19 aluno1 = Aluno('João', 15)
20
21 aluno2 = Aluno('Maria', 16)
```

Objeto

Objeto

```
18 # Atributos do Objeto
19 aluno1 = Aluno('João', 15)
20 print(f"Aluno: {aluno1.nome}, idade: {aluno1.idade}")
21 aluno1.nome = 'Pedro'
22 print(f"Aluno: {aluno1.nome}, idade: {aluno1.idade}")
```

Atributo alterado

```
18 # Métodos do Objeto
19 aluno1 = Aluno('João', 15)
20 aluno1.adicionar_nota(8)
21 aluno1.adicionar_nota(7)
22 print(f'Média: {aluno1.calcular_media()}')
23
24 aluno2 = Aluno('Maria', 16)
25 aluno2.adicionar_nota(9)
26 print(f'Média: {aluno2.calcular_media()}')
```

Métodos do objeto aluno1

Métodos do objeto aluno2

Na prática:

```
#criando uma classe
class ContaBancaria:
    def __init__(self, titular, saldo = 0):
        self.titular = titular
        self.saldo = saldo

#Criando Métodos (ações)
    def depositar (self, valor):
        self.saldo += valor

    def sacar (self, valor):
        if self.saldo >= valor:
            self.saldo -= valor
        else:
            print("Saldo insuficiente")
```

```
#Instanciando Objetos
```

```
Conta1 = ContaBancaria("João", 0)
Conta1.depositar(5)
Conta1.sacar(10)
print(Conta1.saldo)
```

```
Saldo insuficiente
5
```

obs: O método `__init__`, é um método especial denominado construtor da classe. Esse método é chamado automaticamente ao se criar o objeto

Encapsulamento

Conversão é usada para indicar proteção.

```
class ContaBancaria:
    def __init__(self, titular, saldo = 0):
        self.titular = titular
        self.__saldo = saldo #atributo protegido

    def depositar(self, valor):
        self.__saldo += valor

    def sacar(self, valor):
        if self.__saldo >= valor:
            self.__saldo -= valor

    def get_saldo(self):
        return self.__saldo
```

```
Conta1 = ContaBancaria("João", 100)
Conta1.depositar(10)
Conta1.sacar(50)
print(Conta1.get_saldo())
```

60

Outros exemplos:

```
class Carro:
    def __init__(self, modelo):
        self.modelo = modelo

    def ligar(self):
        print(f"O {self.modelo} está ligado")
```

```
carro1 = Carro("Fusca")
carro1.ligar()
```

```
... O Fusca está ligado
```

```
class calculadora:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def somar(self):
        resultado = self.a + self.b
        print(f'A soma de {self.a} e {self.b} é igual a {resultado}')

    def subtrair(self):
        resultado = self.a - self.b
        print(f'A subtração de {self.a} e {self.b} é igual a {resultado}')

    def multiplicar(self):
        resultado = self.a * self.b
        print(f'A multiplicação de {self.a} e {self.b} é igual a {resultado}')

    def dividir(self):
        resultado = self.a / self.b
        print(f'A divisao de {self.a} e {self.b} é igual a {resultado}')

calculadora = calculadora(2, 5)

calculadora.somar()
calculadora.subtrair()
calculadora.multiplicar()
calculadora.dividir()
```